

SKILL SESSION 7&8

Session 7:

TASK 1:

UW PICO 5.09

File: task1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    pid1 = fork();

    if (pid1 < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid1 == 0) {
        printf("Child 1 started. PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 completed.\n");
        exit(10);
    }

    pid2 = fork();

    if (pid2 < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid2 == 0) {
        printf("Child 2 started. PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 completed.\n");
        exit(20);
    }
}
```

```
printf("Parent waiting for Child 1...\n");

waitpid(pid1, &status, 0);

if (WIFEXITED(status)) {
    printf("Child 1 exited with status %d\n",
           WEXITSTATUS(status));
}

printf("Parent waiting for Child 2...\n");

waitpid(pid2, &status, 0);

if (WIFEXITED(status)) {
    printf("Child 2 exited with status %d\n",
           WEXITSTATUS(status));
}

printf("All children completed.\n");
printf("Parent process completed.\n");
```

```
printf("Parent waiting for Child 2...\n");

waitpid(pid2, &status, 0);

if (WIFEXITED(status)) {
    printf("Child 2 exited with status %d\n",
           WEXITSTATUS(status));
}

printf("All children completed.\n");
printf("Parent process completed.\n");

return 0;
```

```
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air Skill4 % clang task1.c -o task1
[(base) saiabburi@Geyasree-MacBook-Air Skill4 % ./task1
Parent process started. PID = 6387
Parent waiting for Child 1...
Child 1 started. PID = 6398
Child 2 started. PID = 6399
Child 2 completed.
Child 1 completed.
Child 1 exited with status 10
Parent waiting for Child 2...
Child 2 exited with status 20
All children completed.
Parent process completed.
(base) saiabburi@Geyasree-MacBook-Air Skill4 %
```

TASK 2:

UW PICO 5.09

File: task2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char *path;
    char *path_copy;
    char *directory;
    char fullpath[PATH_MAX];

    char command[100];

    printf("Enter command to locate: ");
    scanf("%99s", command);

    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    path_copy = malloc(strlen(path) + 1);

    if (path_copy == NULL) {
        perror("malloc failed");
        return 1;
    }

    strcpy(path_copy, path);

    directory = strtok(path_copy, ":");

    while (directory != NULL) {
```

```

while (directory != NULL) {

    snprintf(fullpath, sizeof(fullpath),
             "%s/%s", directory, command);

    if (access(fullpath, X_OK) == 0) {
        printf("Command found: %s\n", fullpath);
        printf("Executable permission verified.\n");

        free(path_copy);
        return 0;
    }

    directory = strtok(NULL, ":");
}

printf("Command '%s' not found in PATH.\n", command);

free(path_copy);

return 1;
}

```

Parent process completed.

```

[(base) saiabburi@Geyasree-MacBook-Air Skill4 % nano task2.c
[(base) saiabburi@Geyasree-MacBook-Air Skill4 % clang task2.c -o task2
[(base) saiabburi@Geyasree-MacBook-Air Skill4 % ./task2
Enter command to locate: ls
Command found: /bin/ls
Executable permission verified.
[(base) saiabburi@Geyasree-MacBook-Air Skill4 % ./task2
Enter command to locate: clang
Command found: /usr/bin/clang
Executable permission verified.
[(base) saiabburi@Geyasree-MacBook-Air Skill4 % ./task2
Enter command to locate: Hello123
Command 'Hello123' not found in PATH.
(base) saiabburi@Geyasree-MacBook-Air Skill4 %

```

Session 8:

TASK 1:

UW PICO 5.09

File: task1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

void expand_variables(char *input, char *output, int size) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < size - 1) {
        if (input[i] == '$') {
            i++;

            char variable[100];
            int k = 0;

            while (input[i] != '\0' &&
                    (isalnum(input[i]) || input[i] == '_')) {
                if (k < 99) {
                    variable[k++] = input[i];
                }
                i++;
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL) {
                while (*value != '\0' && j < size - 1) {
                    output[j++] = *value++;
                }
            }
            else {
                output[j++] = input[i++];
            }
        }
    }
}
```

```

    output[j] = '\0';
}

int main() {
    char input[500];
    char output[1000];

    printf("Enter text with variables: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output, sizeof(output));

    printf("Expanded text: %s\n", output);

    return 0;
}

```

```

[(base) saiabburi@Geyasree-MacBook-Air skill % clang task1.c -o task1
[(base) saiabburi@Geyasree-MacBook-Air skill % ./task1
Enter text with variables: My home is $HOME
Expanded text: My home is /Users/saiabburi
[(base) saiabburi@Geyasree-MacBook-Air skill % ./task1
Enter text with variables: Current user is $USER
Expanded text: Current user is saiabburi
[(base) saiabburi@Geyasree-MacBook-Air skill % ./task1
Enter text with variables: Value is $NOTDEFINED
Expanded text: Value is
[(base) saiabburi@Geyasree-MacBook-Air skill % ./task1
Enter text with variables: User: $USER Home: $HOME
Expanded text: User: saiabburi Home: /Users/saiabburi
(base) saiabburi@Geyasree-MacBook-Air skill % 

```

TASK 2:

```
UW PICO 5.09                                     File: task2.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 200

void builtin_pwd(char *args[]) {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
}

void builtin_cd(char *args[]) {
    if (args[1] == NULL) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args[1]) != 0) {
        perror("cd");
    }
}

void builtin_echo(char *args[]) {
    int i = 1;

    while (args[i] != NULL) {
        printf("%s", args[i]);

        if (args[i + 1] != NULL) {
            printf(" ");
        }

        i++;
    }
}
```

```
printf("Mini Shell\n");
printf("Built-ins: pwd, cd, echo, exit\n");

while (1) {

    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        continue;
    }

    char *args[20];
    int count = 0;

    char *token = strtok(input, " ");

    while (token != NULL && count < 19) {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (strcmp(args[0], "exit") == 0) {
        printf("Exiting shell...\n");
        break;
    }

    else if (strcmp(args[0], "pwd") == 0) {
        builtin_pwd(args);
    }

    args[count] = NULL;

    if (strcmp(args[0], "exit") == 0) {
        printf("Exiting shell...\n");
        break;
    }

    else if (strcmp(args[0], "pwd") == 0) {
        builtin_pwd(args);
    }

    else if (strcmp(args[0], "cd") == 0) {
        builtin_cd(args);
    }

    else if (strcmp(args[0], "echo") == 0) {
        builtin_echo(args);
    }

    else {
        printf("Invalid command: %s\n", args[0]);
    }

    return 0;
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air skill % nano task2.c
[(base) saiabburi@Geyasree-MacBook-Air skill % clang task2.c -o task2
[(base) saiabburi@Geyasree-MacBook-Air skill % ./task2
Mini Shell
Built-ins: pwd, cd, echo, exit
myshell> PWD
Invalid command: PWD
myshell> pwd
/Users/saiabburi/Skill
myshell> echo Hello Charishma
Hello Charishma
myshell> cd ..
myshell> pwd
/Users/saiabburi
myshell> abcxyz
Invalid command: abcxyz
myshell> cd somethingthatdoesnotexist
cd: No such file or directory
myshell> exit
Exiting shell...
(base) saiabburi@Geyasree-MacBook-Air skill %
```

PRACTICE PROBLEMS

TASK 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
□
#define MAX_TOKENS 50
#define MAX_LENGTH 100

typedef struct {
    char value[MAX_LENGTH];
} Token;

int main() {
    char input[500];
    Token tokens[MAX_TOKENS];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS) {
        strcpy(tokens[count].value, token);
        count++;
        token = strtok(NULL, " \t");
    }

    printf("\n--- Token Stream ---\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i].value);
    }

    printf("\n--- Parse Structure ---\n");
```

```
printf("\n--- Parse Structure ---\n");

if (count > 0) {
    printf("Command: %s\n", tokens[0].value);

    if (count > 1) {
        printf("Arguments:\n");

        for (int i = 1; i < count; i++) {
            printf("    Argument %d: %s\n",
                i, tokens[i].value);
        }
    }

    printf("\nParsing completed successfully.\n");

    return 0;
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air Skil % clang task1.c -o task1
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task1
myShell> echo Hello

--- Token Stream ---
Token 1: echo
Token 2: Hello

--- Parse Structure ---
Command: echo
Arguments:
  Argument 1: Hello

Parsing completed successfully.
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task1
myShell> ls -l test.txt

--- Token Stream ---
Token 1: ls
Token 2: -l
Token 3: test.txt

--- Parse Structure ---
Command: ls
Arguments:
  Argument 1: -l
  Argument 2: test.txt

Parsing completed successfully.
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task1
myShell>
Empty command.
[(base) saiabburi@Geyasree-MacBook-Air Skil % █
```

TASK 2: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[500];
    int single_quotes = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\') {
            single_quotes++;
        }
    }

    if (single_quotes % 2 != 0) {
        printf("Syntax Error: Unmatched single quote.\n");
        return 1;
    }

    printf("\nParsing Result:\n");

    if (strncmp(input, "echo ", 5) == 0) {
        char *text = input + 5;

        if (text[0] == '\\' &&
            text[strlen(text) - 1] == '\\') {
            text[strlen(text) - 1] = '\0';
            text++;
        }
    }
}
```

```
        printf("Command: echo\n");
        printf("Quoted Argument: %s\n", text);
        printf("Variable expansion ignored.\n");
    }
    else {
        printf("Command: echo\n");
        printf("Argument: %s\n", text);
    }
}
else {
    printf("Input parsed successfully.\n");
}

return 0;
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air Skil % nano task2.c
[(base) saiabburi@Geyasree-MacBook-Air Skil % clang task2.c -o task2
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task2
myShell> echo 'Hello World'

Parsing Result:
Command: echo
Quoted Argument: Hello World
Variable expansion ignored.
[(base) saiabburi@Geyasree-MacBook-Air Skil % USER_NAME="Charishma"
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task2
myShell> echo '$USER_NAME'

Parsing Result:
Command: echo
Quoted Argument: $USER_NAME
Variable expansion ignored.
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task2
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
(base) saiabburi@Geyasree-MacBook-Air Skil %
```

TASK 3: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char input[500];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    int quotes = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '"') {
            quotes++;
        }
    }

    if (quotes % 2 != 0) {
        printf("Syntax Error: Unmatched double quote.\n");
        return 1;
    }

    printf("--- Parsed Arguments ---\n");

    if (strncmp(input, "echo ", 5) == 0) {
        char *text = input + 5;

        printf("Argument 1: echo\n");

        if (text[0] == '"' &&
            text[strlen(text) - 1] == '"') {
            text[strlen(text) - 1] = '\0';
            text++;
        }
    }
}
```

```
char expanded[500];

char *home = getenv("HOME");
char *user = getenv("USER");

if (strstr(text, "$HOME") != NULL && home != NULL) {
    char temp[500];

    char *position = strstr(text, "$HOME");

    int before = position - text;

    snprintf(temp, sizeof(temp),
             "%.s%s%s",
             before,
             text,
             home,
             position + 5);

    strcpy(text, temp);
}

if (strstr(text, "$USER") != NULL && user != NULL) {
    char temp[500];

    char *position = strstr(text, "$USER");

    int before = position - text;
}
```

```

        snprintf(temp, sizeof(temp),
                 "%. *s%s%s",
                 before,
                 text,
                 home,
                 position + 5);

        strcpy(text, temp);
    }

    if (strstr(text, "$USER") != NULL && user != NULL) {
        char temp[500];

        char *position = strstr(text, "$USER");

        int before = position - text;

        snprintf(temp, sizeof(temp),
                 "%. *s%s%s",
                 before,
                 text,
                 user,
                 position + 5);

        strcpy(text, temp);
    }

    strcpy(expanded, text);

    printf("Argument 2: %s\n", expanded);
}
else {
    printf("Argument 2: %s\n", text);
}
}

return 0;
}

```

```

(base) saiabburi@Geyasree-MacBook-Air Skil % nano task3.c
(base) saiabburi@Geyasree-MacBook-Air Skil % clang task3.c -o task3
(base) saiabburi@Geyasree-MacBook-Air Skil % ./task3
myShell> echo "Hello World"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
(base) saiabburi@Geyasree-MacBook-Air Skil % ./task3
myShell> echo "Hello $USER"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello saiabburi
(base) saiabburi@Geyasree-MacBook-Air Skil % ./task3
myShell> echo "Hello World"
Syntax Error: Unmatched double quote.
(base) saiabburi@Geyasree-MacBook-Air Skil % █

```

TASK 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

UW PICO 5.09

File: task4.c

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[500];
    char output[500];

    int i = 0;
    int j = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0' && j < 499) {
        if (input[i] == '\\') {
            i++;

            if (input[i] != '\0') {
                output[j++] = input[i++];
            }
        }
        else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';

    printf("Parsed Output: %s\n", output);

    return 0;
}
```

```

(base) saiabburi@Geyasree-MacBook-Air Skil % nano task4.c
(base) saiabburi@Geyasree-MacBook-Air Skil % clang task4.c -o task4
(base) saiabburi@Geyasree-MacBook-Air Skil % ./task4
myShell> echo Hello\ World
Parsed Output: echo Hello World
(base) saiabburi@Geyasree-MacBook-Air Skil % ./task4
myShell> echo Hello\!\ \ $USER
Parsed Output: echo Hello! $USER
(base) saiabburi@Geyasree-MacBook-Air Skil % ./task4
myShell> echo Hello\ World\!\ \ #Test
Parsed Output: echo Hello World! #Test
(base) saiabburi@Geyasree-MacBook-Air Skil %

```

TASK 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

UW PICO 5.09 File: task5.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {

    if (argc < 2) {
        printf("Usage: %s <command> [arguments...]\n", argv[0]);
        return 1;
    }

    printf("Parent Process PID: %d\n", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {

        printf("Child Process PID: %d\n", getpid());

        printf("Executing command: %s\n", argv[1]);

        execvp(argv[1], &argv[1]);

        perror("Command execution failed");
        exit(127);
    }

    else {

```

```

    else {
        int status;

        printf("Created Child PID: %d\n", pid);

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
        else {
            printf("Child did not exit normally.\n");
        }
    }

    return 0;
}

```

```

[(base) saiabburi@Geyasree-MacBook-Air Skil % nano task5.c
[(base) saiabburi@Geyasree-MacBook-Air Skil % clang task5.c -o task5
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task5 ls
Parent Process PID: 7377
Created Child PID: 7388
Child Process PID: 7388
Executing command: ls
task1  task1.c task2  task2.c task3  task3.c task4  task4.c task5  task5.c
Child exited with status: 0
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task5 echo Hello Charishma
Parent Process PID: 7389
Created Child PID: 7390
Child Process PID: 7390
Executing command: echo
Hello Charishma
Child exited with status: 0
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task5 wrongcommand
Parent Process PID: 7394
Created Child PID: 7395
Child Process PID: 7395
Executing command: wrongcommand
Command execution failed: No such file or directory
Child exited with status: 127
[(base) saiabburi@Geyasree-MacBook-Air Skil % ./task5 ls @1
Parent Process PID: 7398
Created Child PID: 7399
Child Process PID: 7399
Executing command: ls
ls: @1: No such file or directory
Child exited with status: 1
[(base) saiabburi@Geyasree-MacBook-Air Skil %

```

